Estimating word-probabilities using SRILM

Sidharth Ranjan

1 Introduction

In the previous class, we counted the occurrence of each word present in English `Toy` dataset and estimated their word probabilities. As we discussed, this approach is challenging for large corpus containing millions of words. Our goal in this exercise is to automate the computation and estimate the probabilities of words in a large corpus using open-source software – `SRI Language Modeling Toolkit` (SRILM). Please read the documentation available on SRILM website and install the toolkit on your Linux system. The main functionalities of SRILM toolkit are outlined below (see Figure 1):

Step 1: Generate n-gram count file from a text corpus

Step 2: Train a *language model* from n-gram count file obtained in Step 1 using smoothing and back-off probabilities

Step 3: Using the trained language model obtained in Step 2, estimate the perplexity of the test data and per-word log probabilities (likelihood) of the word sequences

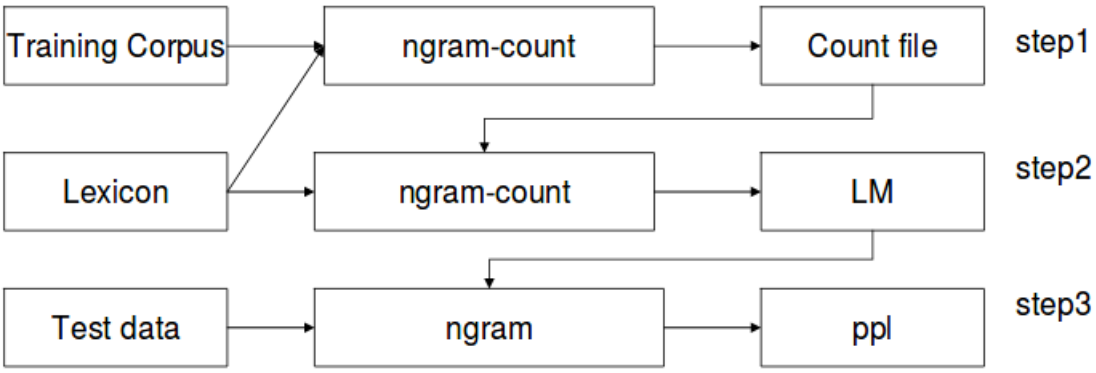


Figure 1: n-gram language modeling

2 Download dataset and SRILM binaries

In order to get started with this tutorial, please download the following items from this link. You should unzip the compressed files and place it in your working directory.

1. Brown Corpus: `brown-train.txt, brown-dev.txt, brown-test.txt`
2. SRILM Binary executable files: `ngram-count, ngram`

3 Word Counting

This section deals with Step 1 procedure as shown in Figure 1. Here, we estimate the n-gram counts from the training data by using `ngram-count` binary file you just downloaded. You should navigate your present working directory to the folder containing binary executable files and run the following command in the terminal.

```
./ngram-count -text browndata/brown-train.txt -order 2 -write brown-train.count
```

The above command makes a bigram (n = 2) count file with name `brown-train.count` in your work directory. Please try cross-checking the counts obtained for each word present in `brown-train.txt` file using some UNIX command (if possible). SRILM assumes $< s >$ and $< /s >$ as a beginning and ending tokens respectively for every sentence in your training dataset. Moreover, it also ignores any empty lines present in your sentence file.

4 Language Modeling

Now that we are ready with count file, we shall workout Step 2 as shown in Figure 1 and create a language model (`.lm`) file. SRILM considers `Good Turing discounting` smoothing technique as a pre-selected option unless otherwise specified. However, if you want an un-smoothed per-word probabilities, specify `-addsmooth 0` as argument in your command. The `brown-train.count` file created in Section 3 shall be used in generating un-smoothed (`.lm`) file as below:

```
./ngram-count -read brown-train.count -order 2 -lm brown-train.lm -addsmooth 0 -unk
```

The `-unk` argument ensures that out-of-vocabulary (OOV) words during testing process shall be annotated with UNKNOWN tag and smoothed probability. The above command creates a language model file with the name `brown-train.lm` in your work directory. This file contains backed-off and n-grams probabilities in `ARPA` format which shall be used to estimate word probabilities of unknown test data. Read the documentation here for more details. You can also combine these two separate processes (*count* and *lm*) into one step process and directly get `.count` and `.lm` file by running below command:

```
./ngram-count -text browndata/brown-train.txt -order 2 -write brown-train.count -unk -lm
brown-train.lm -addsmooth 0
```

5 Testing: Perplexity and Word Probability Estimation

We are now ready to explore Step 3 in Figure 1 i.e., test on unknown dataset assuming you have `brown-train.count` and `brown-train.lm` in your work directory. For this, we shall use `ngram` executable binary to compute perplexity and per-word probability score for the test sentences using `-ppl` and `-debug 2` arguments respectively.

For calculating perplexity on the testing dataset:

```
./ngram -lm brown-train.lm -ppl browndata/brown-test.txt
```

For calculating the per-word probability of each testing sentence:

```
./ngram -lm brown-train.lm -ppl browndata/brown-test.txt -debug 2
```

6 Some more exercises!

1. Why did you get `-inf` log-probability for OOV words above? How can you fix it? What do `-unk` and `-addsmooth 0` signify?
2. Use a different smoothing algorithm other than `-addsmooth` that has been discussed in class during Step 2 and observe the perplexity score and out of vocabulary (OOV) probabilities?
3. Analyze the output of your LM model by varying n (say n = 3), i.e., consider training and testing of a trigram LM model?
4. How does your model prediction get affected if you pre-process your training and testing data with operations viz., punctuation removal or use of STOP words?
5. Can you train an interpolated model such as combination of *bigram* and *trigram* language models? [Hint : Use `-mix-lm` and `-lamda` (Read here!)]
   a) Train a separate `unigram` and `bigram` LM model with small values of `-addsmooth`
   b) Jointly use both these trained models obtained above to test on your development set `brown-dev.txt` as indicated in Step 3 of Figure 1. One can use `-mix-lm` argument and `-lamda` ***weight*** parameter for the same.
   c) Find out best `lamba` ϵ [0,1] (also known as interpolation weight) and `-addsmooth` parameters for which you get less perplexity on your development set. This process is also known as `hyperparameter tuning`.
   d) Finally, use the optimum values of `-addsmooth` and `-lamda` parameter obtained during `hyperparameter tuning` to test on the unseen dataset — `brown-test.txt`

7 Extra Reading Materials

- http://www.speech.sri.com/projects/srilm/manpages/
- http://www.speech.sri.com/projects/srilm/papers/icslp2002-srilm.pdf
- https://web.stanford.edu/~jurafsky/slp3/3.pdf
- https://nlp.stanford.edu/fsnlp/statest/
- http://www.speech.sri.com/projects/srilm/papers/asru2011-srilm.pdf